

Week 4: TSP Introduction

1. Quick Graph Recap

Before diving into Hamiltonian cycles and TSP, make sure the following vocabulary is rock-solid. These ideas will appear constantly in competitive programming.

Term	Definition	Example
Vertex (node)	A point/city in the graph	City A, City B ...
Edge	A connection between two vertices	Road from A to B
Weight	Cost/distance assigned to an edge	Distance = 42 km
Degree	Number of edges touching a vertex	City A has 3 roads
Path	Sequence of vertices, no repeated vertex	A -> B -> C -> D
Cycle	A path that starts and ends at the same vertex	A -> B -> C -> A
Complete graph K_n	Every pair of vertices is connected	K_4 has 6 edges
Connected graph	Path exists between every pair of vertices	All cities reachable

2. Eulerian Cycle

2.1 Formal Definition

An **Eulerian cycle** (also called an Eulerian circuit) is a cycle in a graph that **traverses every edge exactly once** and returns to the starting vertex. Vertices may be revisited — it is edges that must each appear exactly once.

Definition – Eulerian Cycle:

A cycle that uses every edge of the graph exactly once.
Vertices can be visited more than once.

Existence condition (undirected graph):

The graph is connected AND every vertex has even degree.

Existence condition (directed graph):

The graph is strongly connected AND
 $\text{in-degree}(v) = \text{out-degree}(v)$ for every vertex v .

2.2 The Famous Origin — Königsberg Bridge Problem

In 1736, Leonhard Euler solved the Königsberg Bridge Problem: can you walk through the city crossing each of its 7 bridges exactly once and return to your starting point? Euler proved it was impossible because the graph had vertices with odd degree. This was the birth of graph theory itself. The key insight: an Eulerian cycle exists if and only if every vertex has even degree (in an undirected connected graph).

2.3 Finding an Eulerian Cycle — Hierholzer's Algorithm

Once you know an Eulerian cycle exists, finding it is efficient:

Hierholzer's Algorithm:

1. Start at any vertex, follow edges (removing them) until you return to start.

This gives a partial cycle C .

2. If C uses all edges: done.

3. Otherwise, find a vertex in C that still has unused edges.

4. Start a new sub-cycle from that vertex, splice it into C .

5. Repeat until all edges are used.

Time Complexity: $O(E)$ — incredibly fast, linear in number of edges!

The $O(E)$ complexity of Eulerian cycle detection and construction stands in stark contrast to the exponential complexity of Hamiltonian cycle problems — this difference is one of the most striking examples of how similar-sounding problems can have wildly different complexities.

3. Hamiltonian Cycle

3.1 Formal Definition

A **Hamiltonian cycle** (also called a Hamiltonian circuit) is a cycle that **visits every vertex exactly once** and returns to the starting vertex. Unlike Eulerian cycles, it is edges that may be skipped — every vertex must appear exactly once.

Definition — Hamiltonian Cycle:

Given a graph G with n vertices, a Hamiltonian cycle is a sequence $v_0, v_1, v_2, \dots, v_{(n-1)}, v_0$ such that:

- (1) Each v_i is a distinct vertex of G
- (2) Each consecutive pair (v_i, v_{i+1}) is an edge in E
- (3) The pair (v_{n-1}, v_0) is also an edge in E (closing the cycle)

There is NO simple polynomial-time test for existence (it is NP-complete).

3.2 Sufficient Conditions (Not Necessary)

- **Dirac (1952)**: If every vertex has degree $\geq n/2$, a Hamiltonian cycle exists.
- **Ore (1960)**: If $\deg(u) + \deg(v) \geq n$ for every non-adjacent pair, a Ham. cycle exists.
- **Complete graphs K_n ($n \geq 3$)**: Always have a Hamiltonian cycle.

3.3 Hamiltonian Path vs. Hamiltonian Cycle

Property	Hamiltonian Path	Hamiltonian Cycle
Visits all vertices?	Yes	Yes
Returns to start?	No	Yes
Edges used	$n - 1$	n
Application	Route planning (one-way)	TSP, circular routes

4. Eulerian vs. Hamiltonian — The Critical Difference

These two cycle types are often confused by beginners. They sound similar but are fundamentally different in what they require and how hard they are to solve.

Property	Eulerian Cycle	Hamiltonian Cycle
What must appear exactly once?	Every EDGE	Every VERTEX
Can edges be skipped?	No	Yes
Can vertices be revisited?	Yes	No
Existence: easy to check?	YES — $O(V+E)$ via degree check	NO — NP-complete
Finding one (if exists)?	YES — $O(E)$ Hierholzer	NO — exponential (brute force)
Key constraint	Even degree at all vertices	No simple polynomial condition

Complexity class	P (polynomial time)	NP-complete
------------------	---------------------	-------------

4.1 A Concrete Example

Consider a square graph: vertices A, B, C, D connected in a cycle A-B-C-D-A with one diagonal B-D added.

- Eulerian cycle: Does one exist? Degrees: A=2, B=3, C=2, D=3. Vertex B and D have odd degree. NO Eulerian cycle.
- Hamiltonian cycle: A → B → C → D → A visits all 4 vertices exactly once. YES, Hamiltonian cycle exists.
- Same graph, opposite answers — this illustrates they are independent properties!

5. The Travelling Salesman Problem (TSP)

5.1 Problem Statement

TSP – Formal Statement:
Given a complete weighted graph $G = (V, E, w)$ where $w(u,v)$ is the cost of edge (u,v) ,
find a Hamiltonian cycle of minimum total weight.

Input: n cities, pairwise distances $d[i][j]$
Output: A permutation of cities (a tour) that visits each city exactly once
and returns to start, minimising the total travel cost.

5.2 Variants of TSP

Variant	Key Constraint	Notes
Symmetric TSP	$d[i][j] = d[j][i]$	Most common; undirected graph
Asymmetric TSP	$d[i][j]$ may not equal $d[j][i]$	Directed graph (one-way roads)
Metric TSP	Triangle inequality holds	Allows approximation guarantees
Euclidean TSP	Cities are points in 2-D plane	Special case of metric TSP

5.3 Worked Example (n = 4)

Cities: 0 (Home), 1, 2, 3. Distance matrix:

	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

Tour	Cost Calculation	Total
0->1->2->3->0	10 + 35 + 30 + 20	95
0->1->3->2->0	10 + 25 + 30 + 15	80 <- OPTIMAL
0->2->1->3->0	15 + 35 + 25 + 20	95

6. Why is TSP Hard? NP-Hardness

6.1 Brute Force

Fix city 0 as start. Try all $(n-1)!$ permutations of the remaining cities.

Time Complexity: $O(n!)$ -- catastrophically slow

Space Complexity: $O(n)$

n (cities)	Permutations checked	Approx. time @ 10^9 ops/sec
5	24	< 1 microsecond
10	362,880	< 1 millisecond
15	87,178,291,200	~87 seconds
20	121,645,100,408,832,000	~3.8 million years
25	$\sim 6.2 \times 10^{23}$	Longer than the universe's age

Key insight: Even doubling your CPU speed only lets you handle ~1 extra city before hitting the same wall. Factorial growth completely dominates any hardware improvement.

6.2 NP-Hardness and NP-Completeness

To accurately describe the complexity of TSP, we must distinguish between its two forms: the **Decision Version** and the **Optimization Version**. The way we classify their difficulty relies on understanding P, NP, NP-Complete, and NP-Hard.

Complexity Classes Refresher:

- P: Problems that can be SOLVED in polynomial time.
- NP: Problems where a proposed solution can be VERIFIED in polynomial time.
- NP-Hard: Problems that are 'at least as hard as the hardest problems in NP'.
- NP-Complete: Problems that are BOTH in NP and are NP-Hard.

The Decision Version (NP-Complete)

Question: 'Given a graph and a budget K , does there exist a tour with a total cost $\leq K$?'

This version is **NP-Complete**. First, it belongs to NP: if someone hands you a sequence of cities, you can easily add up the edge weights in $O(n)$ time and verify if the sum is indeed $\leq K$. Second, it is NP-Hard because any other problem in NP can be mathematically translated

(reduced) into this problem in polynomial time.

The Optimization Version (NP-Hard)

Question: 'What is the absolute minimum cost Hamiltonian cycle in the graph?'

This version is **NP-Hard**, but technically NOT in NP. Why? Because if someone hands you a tour and claims 'This is the absolute shortest possible route', there is no quick (polynomial time) way to verify that claim. To prove it is the optimal tour, you would theoretically have to compare it against other tours. It is 'at least as hard' as the decision version, but it lacks the quick verification property of NP.

Conclusions:

- **No known fast exact algorithm:** Nobody has found a way to solve all TSP instances optimally in polynomial time (like $O(n^2)$ or $O(n^3)$).
- **P vs NP:** It is widely believed by computer scientists that $P \neq NP$, meaning such a fast, exact algorithm likely does not exist.
- **Worst-case vs Average-case:** 'NP-Hard' describes the absolute worst-case scenario. Many real-world instances have specific structures (like metric or Euclidean properties) that allow solvers to find exact answers quickly despite the theoretical worst-case classification.

7. Some Exact Algorithms (Optional)

7.1 Held-Karp: Bitmask DP

The Held-Karp algorithm (1962) uses dynamic programming with a bitmask to represent which cities have been visited. It is the classic exact solution for small-to-medium TSP.

State Definition

$dp[mask][i]$ = minimum cost to reach city i , having visited exactly the cities indicated by the bits in mask (bit j set = city j visited)

mask: an integer from 0 to $2^n - 1$ (n bits, one per city)

i : the current city (last city visited)

Base case: $dp[1][0] = 0$ (start at city 0, only city 0 visited)

Answer: $\min \text{ over all } i \text{ of } \{ dp[(1<n)-1][i] + dist[i][0] \}$

Recurrence

To fill $dp[mask][i]$, consider all possible previous cities j (j is in mask, $j \neq i$):

$dp[mask][i] = \min \text{ over all valid } j \{ dp[mask \wedge (1<i)][j] + dist[j][i] \}$

Complexity

- States: 2^n masks $\times$ n cities = $n \times 2^n$ total states
- Transitions: For each state, check up to n previous cities
- Total time: $O(n^2 \times 2^n)$ -- much better than $O(n!)$
- Space: $O(n \times 2^n)$

For $n = 20$: $n!$ is about 2.4×10^{18} vs $n^2 \times 2^n$ is about 4×10^8 operations. Held-Karp is roughly 10 billion times faster for $n = 20$.

7.2 Branch and Bound

Branch and Bound algorithms explore the search space (like a tree of partial tours) and maintain a 'lower bound' on the cost of completing a partial tour. If a partial tour's lower bound exceeds the best complete tour found so far, that branch is entirely pruned. This drastically reduces the search space compared to brute force, though worst-case time is still exponential.

7.3 Integer Linear Programming (ILP) & Branch-and-Cut

TSP can be formulated mathematically as an ILP problem (using subtour elimination constraints like the Miller-Tucker-Zemlin formulation). Advanced commercial solvers like Concorde use cutting-plane methods combined with Branch and Bound (called Branch-and-Cut) to solve massive TSP instances with thousands of cities exactly.

8. Advanced Algorithmic Paradigms to Explore (Optional)

8.1 Deep Dive: What is an Approximation Algorithm?

When a problem is NP-hard, finding the exact optimal solution in polynomial time is considered impossible. An **Approximation Algorithm** is a deterministic algorithm that runs in **polynomial time** and provides a mathematically provable guarantee on the quality of the solution compared to the true optimal solution.

This quality guarantee is expressed as an **approximation ratio** (ρ). For a minimization problem like TSP, if the algorithm produces a tour of cost C and the absolute minimum cost is C^* , the algorithm guarantees: $C \leq \rho * C^*$. For example, a '1.5-approximation' means the generated path will never be more than 50% longer than the best path possible, no matter how chaotic or massive the graph is.

8.2 Brief Overview of Approximation Schemes

2-Approximation via MST: Constructs a Minimum Spanning Tree, runs a DFS traversal, and shortcuts duplicates. It guarantees a tour cost $\leq 2x$ optimal in graphs satisfying the triangle inequality.

Christofides Algorithm: Combines an MST with a minimum-weight perfect matching on odd-degree nodes to form an Eulerian subgraph. It provides an exceptional 1.5x optimality guarantee in polynomial time.

8.3 Brief Overview of Heuristic Paradigms

Unlike approximation algorithms, **Heuristics** and **Local Search** methods don't provide rigorous provable guarantees. However, they are incredibly fast and produce near-optimal solutions in practice.

Strategy Type	Core Heuristic Idea	Typical Performance
Nearest Neighbour	Greedy method: start anywhere and always jump to the nearest unvisited city.	Fast $O(n^2)$, yields paths roughly 20-25% above optimal.
2-opt Local Search	Iteratively alters a path by swapping pairs of edges to untangle crossings.	Extremely popular; routinely gets within 5% of optimal.
Lin-Kernighan (LK)	An advanced local search that dynamically changes the number of edge swaps per pass.	The practical gold standard; typically finishes within 1-2% of optimal.

8.4 Paradigm & Complexity Matrix

Algorithm	Type Paradigm	Time Complexity	Quality Guarantee
-----------	---------------	-----------------	-------------------

Brute Force	Exact (Permutations)	$O(n!)$	Optimal (Exact)
Held-Karp DP	Exact (Bitmask DP)	$O(n^2 \cdot 2^n)$	Optimal (Exact)
Branch & Bound	Exact (Search & Prune)	Exponential (Varies)	Optimal (Exact)
ILP (Branch-and-Cut)	Exact (Optimization)	Exponential (Varies)	Optimal (Exact)
MST 2-Approx	Approximation	$O(n^2)$	Within 2.0x OPT (Strict bound)
Christofides	Approximation	$O(n^3)$	Within 1.5x OPT (Strict bound)
Nearest Neighbour	Greedy Heuristic	$O(n^2)$	No provable bound (Practical execution)
2-opt Search	Local Search Heuristic	$O(n^2)$ per pass	No provable bound (Near-optimal practical)

9. Real-World Applications

TSP and its variants appear across an enormous range of industries. Understanding the problem deeply helps you recognise when a real situation is reducible to TSP.

9.1 Logistics and Last-Mile Delivery

This is the most direct application. Companies like UPS, FedEx, DHL, and Amazon Logistics route tens of thousands of delivery vehicles every day. UPS's ORION system uses a variant of TSP with time windows (TSPTW) to optimise 55,000 routes daily for 66,000 drivers.

9.2 Food Delivery and Hyper-Local Logistics

Food delivery platforms like Swiggy, Zomato, DoorDash, and UberEats face real-time, highly dynamic variations of the TSP. Delivery partners must pick up multiple orders from different restaurants and deliver them to various customer locations while minimizing total travel time and ensuring the food stays hot. This is mathematically modeled as a dynamic Vehicle Routing Problem with Time Windows (VRPTW).

9.3 Printed Circuit Board (PCB) Manufacturing

When manufacturing a circuit board, a drill must punch thousands of tiny holes. The drill head must visit each hole exactly once. A board with 10,000 holes requires solving a 10,000-city Euclidean TSP instance.

9.4 Genome Sequencing and Bioinformatics

In DNA sequencing, a genome is broken into millions of short 'read' fragments. Reassembling these into the original sequence involves finding the shortest superstring that contains all fragments — this problem is NP-hard and is closely related to TSP.

Domain	Specific Problem	Scale in Practice
Package Delivery	UPS/FedEx daily routing (TSPTW)	55,000 routes, 10M+ stops/day
Food Delivery Apps	Dynamic VRPTW (Pickups/Drop-offs)	Millions of orders, real-time recalculation
PCB Manufacturing	Drill head path planning	1,000 - 100,000 holes per board
Genome Assembly	Shortest superstring / Hamiltonian path	Millions of reads per genome
Telescope Scheduling	Minimise slewing time between targets	Hundreds of targets per night
Warehouse Robots	Amazon Kiva multi-shelf item collection	Millions of items, 1000s of robots

9.5 The Chinese Postman Problem — Eulerian in Practice

The Chinese Postman Problem (also called Route Inspection) asks: given a graph representing streets, find the shortest walk that traverses every edge at least once and returns to start. Real applications include: street sweeper routing, snow plough routing, and mail carrier routing.

10. External Resources

Theoretical Deep Dives

- **Christofides Algorithm:** Essential reading for understanding 1.5x approximation guarantees in metric graphs.
- **Lin-Kernighan Heuristic (LKH):** Research the variable-depth K-opt swaps that power the world's most successful heuristic TSP solvers.

Industry-Grade Solvers

- **Google OR-Tools:** An open-source software suite by Google for optimization. Excellent for Vehicle Routing Problems (VRP) and TSP.
- **Concorde TSP Solver:** The undisputed champion of exact TSP solvers, famous for solving an 85,900-city instance optimally.